

A

Mini Project Report

on

DocSafe – Document Forgery Detection System

Submitted in partial fulfillment of the requirements for the
degree

Third Year Engineering – Computer Science Engineering (Data Science)

by

Prasad Jadhav 23107097

Dhananjay Agarwal 23107121

Shardul Kadam 23107068

Riddhi Manchekar 23107109

Under the guidance of

Mrs. Hardiki Achrekar



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on **DocSafe – Document Forgery Detection System** has been submitted by **Prasad Jadhav (23107097)**, **Dhananjay Agarwal (23107121)**, **Shardul Kadam (23107068)** and **Riddhi Manchekar (23107109)** who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Mrs. Hardiki Achrekar
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane

Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Mrs. Hardiki Achrekar**. Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Mrs. Aishwarya Londhe** who gave us his/her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

Abstract

1. Introduction.....	1
1.1.Purpose.....	1
1.2.Problem Statement.....	3
1.3.Objectives.....	5
1.4.Scope.....	7
2. Literature Review.....	9
3. Proposed System.....	11
3.1. Features and Functionality.....	13
4. Requirements Analysis.....	14
5. Project Design.....	17
5.1.System Architecture.....	17
5.2.Use Case Diagram.....	19
5.3.DFD (Data Flow Diagram)	22
5.4.Implementation.....	26
6. Technical Specification.....	27
7. Project Scheduling.....	32
8. Results.....	34
9. Conclusion.....	35
10. Future Scope.....	36

References

ABSTRACT

The rapid digitization of vital records has improved efficiency across administrative, legal, and financial sectors, but it has also introduced significant security vulnerabilities. Advanced digital manipulation tools, including generative AI and professional editing software, now enable highly sophisticated document forgery. Traditional forensic methods—relying on manual inspection or basic image-processing techniques—are inadequate against such modern, multi-layered tampering.

To address this gap, this project presents an automated Document Forgery Detection System based on a hybrid feature extraction approach. Unlike unimodal systems, it analyzes both visual content and structural text layout using Computer Vision and OCR techniques.

The system extracts 64 visual features using OpenCV and NumPy, including histogram distributions, edge density, texture variance, color statistics, and frequency-domain analysis to detect subtle manipulation artifacts. In parallel, Tesseract OCR generates 16 structural features focused on text alignment, bounding box consistency, and OCR confidence variations to identify typographical irregularities.

This dual-analysis model demonstrates strong performance, achieving 87% accuracy, 86.67% precision, 78% recall, and a 82.11% F1 score across diverse forged document datasets.

Built on a scalable microservices architecture, the system integrates a PyTorch inference engine with a FastAPI backend and a React + Tailwind frontend, enabling real-time, user-friendly document verification. It offers a robust, enterprise-grade solution to combat modern digital document fraud.

Chapter 1

Introduction

The exponential growth of digital documentation across administrative, financial, and legal domains has fundamentally transformed how information is created, stored, and exchanged. However, this transformation has simultaneously introduced critical vulnerabilities, as modern editing tools and AI-driven technologies have made document manipulation increasingly accessible and sophisticated. Ensuring the authenticity of digital documents has therefore become a significant challenge, necessitating the development of intelligent, automated systems capable of detecting even the most subtle forms of forgery.

1.1 Purpose

The rapid digitization of global administrative, legal, and financial processes has fundamentally transformed how individuals and organizations manage information. Documents that once existed purely as physical paper records—ranging from identification cards to complex business contracts—are now predominantly generated, transmitted, and stored electronically. This shift offers unparalleled convenience, speed, and efficiency. However, it also introduces a profound vulnerability: the ease with which digital documents can be altered, fabricated, or forged.

Modern graphic editing software, such as Adobe Photoshop, CorelDRAW, and increasingly sophisticated Artificial Intelligence (AI) generation tools, have democratized the ability to manipulate pixels and text. What was once the domain of highly skilled counterfeiters is now accessible to almost anyone with a basic understanding of consumer-grade software. This accessibility has led to a significant increase in document fraud, encompassing activities like identity theft, financial scams, academic fraud, and the falsification of legal evidence.

The traditional methods of verifying document authenticity rely heavily on human visual inspection, which is inherently flawed. Human operators, regardless of their training, are susceptible to fatigue, cognitive bias, and simple visual limitations when it comes to detecting sub-pixel manipulations, microscopic compression artifacts, or subtle inconsistencies in text spacing. Furthermore, manual inspection is fundamentally unscalable in environments that process thousands of documents daily, such as banking institutions

performing Know Your Customer (KYC) checks, or border control agencies verifying travel documents.

In response to this growing threat, the field of digital forensics has evolved significantly. Early automated attempts at forgery detection focused on isolated image processing techniques, such as Error Level Analysis (ELA) or basic Copy-Move Forgery Detection (CMFD). While these methods provided a foundational toolkit, they often proved inadequate against complex, multi-layered manipulations, particularly those involving scanned text or PDF documents where structural formatting is paramount.

The introduction of Deep Learning and Neural Networks has marked a paradigm shift in forensic analysis. By training models on vast datasets of both pristine and manipulated documents, these systems can learn complex, non-linear patterns that indicate tampering—patterns that are invisible to the naked eye and mathematically too complex for traditional algorithms to define explicitly.

This project, the "Document Forgery Detection System," represents a state-of-the-art approach to this critical problem. It diverges from purely visual-based forensic tools by adopting a hybrid analytical model. Recognizing that forgery in official documents often involves both graphical manipulation (e.g., swapping a photograph) and textual alteration (e.g., changing a date or a financial figure), this system integrates Computer Vision techniques with Optical Character Recognition (OCR).

By extracting exactly 80 distinct features—64 derived from the mathematical properties of the image (such as histograms, edge densities, and frequency domains) and 16 derived from the structural and statistical properties of the OCR-extracted text—the system provides a highly granular and comprehensive view of a document's integrity. These features are subsequently fed into a custom-designed PyTorch Deep Neural Network, which acts as the ultimate arbiter, outputting a calculated binary classification (Authentic vs. Forged) accompanied by a rigorous confidence score.

Coupled with a modern, responsive web interface built on React, and served by a high-performance FastAPI Python backend, the system bridges the gap between advanced forensic science and practical, everyday utility. It is designed to be deployed in real-time, high-volume environments, offering organizations a robust, unyielding first line of defense

1.2 Problem Statement

The core problem this project addresses is the escalating vulnerability of digital and scanned documents to sophisticated, widely available manipulation techniques, which traditional verification processes are fundamentally ill-equipped to combat.

We exist in an environment characterized by the "democratization of forgery." Only a decade ago, convincingly altering an official document required specialized skills, expensive hardware, and considerable time. Today, the widespread availability of high-resolution scanners, advanced image editing software (like Adobe Photoshop), and, most critically, generative AI and deepfake technologies, has drastically lowered the barrier to entry for digital fraud.

This technological reality creates a profound disparity. The tools available to those wishing to commit forgery are advancing at an exponential rate, while the tools utilized by the institutions tasked with verifying these documents have largely remained stagnant, relying heavily on manual, human-centric processes. This disparity manifests in several acute problems:

1. The Inadequacy of Human Inspection: Human vision is evolved to recognize patterns, faces, and threats in the physical world; it is specifically not evolved to detect localized noise variances, inconsistent double JPEG compression artifacts, or subtle, mathematically irregular gradient shifts. When an individual uses the "clone stamp" tool in Photoshop to alter a date on a bank statement, they match the surrounding background color and texture. To a human reviewer spending 30 seconds looking at a PDF on a monitor, the document appears flawless. The problem is that manual review is subjective, prone to fatigue, and easily defeated by competent digital manipulation.

2. The Scale and Velocity of Digital Submissions: Financial institutions, insurance companies, and government portals process thousands, sometimes millions, of document submissions daily. Whether it is KYC (Know Your Customer) verifications, insurance claim attachments, or remote onboarding processes, the sheer volume of data makes thorough manual inspection impossible. Organizations are forced to choose between accepting a high degree of operational risk by sampling only a fraction of documents, or incurring massive

operational costs by employing armies of human reviewers. There is a critical problem in achieving both scale and security simultaneously.

3. The Complexity of Modern Document Formats: Digital documents are not monolithic. A standard PDF might be a purely digital, native document containing vector text; it might be a wrapper around a single, high-resolution scanned image (a raster PDF); or it might be a complex composition of multiple images and text layers. Forgers exploit this complexity. For example, they might extract an image from a native PDF, alter the text within that specific image, and repackage it. Traditional metadata analysis often fails here because the manipulation occurs at the pixel level of the embedded assets, not necessarily in the file's overarching metadata.

4. The Dual Vector of Forgery: A critical problem with existing automated detection systems is their singular focus. Tools designed for photographic forensics (like analyzing lighting inconsistencies in a portrait) are highly ineffective when applied to a scanned text document. Conversely, tools that look purely at text extraction miss visual cut-and-paste jobs. Document forgery frequently involves both vectors: taking a genuine layout (visual) and altering the specific numbers or names (textual). A system that analyzes only one vector is severely handicapped. The problem is the lack of a unified, hybrid approach that treats a document as both a structured text container and a complex visual matrix.

Therefore, the defining problem the Document Forgery Detection System solves is the critical need for an automated, scalable, and highly accurate technology that can counter sophisticated digital tampering by simultaneous, microscopic analysis of both visual and textual document features, removing the burden from fallible human inspection.

1.3 Objectives

To systematically address the sophisticated challenges posed by modern document manipulation, this project is guided by a rigorous, foundational set of actionable objectives. These four core objectives govern the entire processing pipeline, outlining the journey from raw pixel data to authoritative algorithmic classification and continuous statistical evaluation.

Objective 1: To preprocess images using greyscale conversion and enhancement to highlight forgery artifacts

The system first standardizes all input documents using grayscale conversion to eliminate unnecessary color noise and highlight structural details. This transformation simplifies the data into a luminance-based representation, exposing anomalies such as contrast inconsistencies and artificial shadows.

Further enhancement techniques—such as Gaussian blurring and edge detection—are applied to suppress natural noise while amplifying forgery artifacts like compression traces and pixel interpolation distortions. This ensures that feature extraction operates on optimized, high-quality data.

Objective 2: To extract textual content using Optical Character Recognition (OCR) and analyze inconsistencies for tampering detection

Beyond pixel-level inspection, the system analyzes the semantic and geometric structure of text using OCR. Instead of focusing solely on text extraction, it evaluates spatial metadata such as bounding boxes and alignment patterns.

Forgery often disrupts the natural typographical layout of a document. By analyzing inconsistencies in spacing, word dimensions, and OCR confidence levels, the system identifies subtle tampering that may appear visually seamless but is structurally flawed.

Objective 3: To classify documents as forged or genuine using Support Vector Machine and Random Forest algorithms.

The extracted visual and textual features are combined into a unified feature vector and classified using two robust machine learning models:

- **Support Vector Machine (SVM):**
Constructs an optimal hyperplane to separate forged and genuine documents in a high-dimensional feature space.
- **Random Forest:**
Uses an ensemble of decision trees to improve robustness and reduce overfitting by aggregating multiple predictions.

This dual-model approach ensures high interpretability, stability, and generalization across diverse document types and forgery techniques.

Objective 4: To evaluate system performance using accuracy, precision, recall, and F1-score metrics.

The system's effectiveness is rigorously evaluated using standard classification metrics:

- **Accuracy:** Overall correctness of predictions
- **Precision:** Minimization of false positives (avoiding misclassification of genuine documents)
- **Recall:** Ability to detect actual forgeries (reducing false negatives)
- **F1-Score:** Balanced measure combining precision and recall

These metrics ensure the system maintains both reliability and practical usability in real-world deployments.

1.4 Scope

The scope of the Document Forgery Detection System defines the boundaries of its current capabilities, the specific types of documents and manipulations it is engineered to analyze, and explicitly acknowledges its current limitations and exclusions. Defining the scope is critical for setting proper operational expectations for deployment.

In-Scope: Supported File Formats The system is designed to accept and process the most universally utilized document formats in digital administration:

- **PDF Documents (.pdf):** The system processes native digital PDFs, rasterized (scanned) PDFs, and multi-layer PDFs. For multi-page PDFs, the current scope restricts analysis to the primary (first) page to maintain the 2-5 second processing objective, as forgery is typically clustered on the primary data page. Python's pdf2image (backed by the Poppler utility) is utilized to render PDFs into high-fidelity matrices for visual analysis.
- **Image Formats (.png, .jpg, .jpeg):** Direct image uploads, such as those generated by mobile phone cameras, flatbed scanners, or screen captures, are fully supported and form the core expected input for identity verification use cases. High-resolution inputs (300+ DPI) are optimal within this scope.

In-Scope: Document Categories The hybrid 80-feature model has been generalized to perform accurately across a wide spectrum of official document categories, including:

- **Identity Documents:** National ID cards, Passports, Driver's Licenses, and official internal corporate or student IDs. The system looks for portrait modifications and manipulated expiration dates.
- **Financial Documents:** Bank statements, tax returns, pay stubs, invoices, and digital receipts. The system relies heavily on OCR features here to detect inconsistent numerical alignment or swapped financial figures.
- **Legal & Educational Documents:** Contracts, notarized affidavits, birth/death/marriage certificates, university transcripts, and academic diplomas. The system analyzes overall structural integrity and texture consistencies inherent in official seals or printed watermarks.

In-Scope: Types of Detectable Forgeries The system is explicitly designed to detect the mathematical signatures of several primary forms of digital manipulation:

- **Digital Manipulation (Photoshopping):** The use of Clone Stamp tools, digital erasing, text insertion via text boxes, and layer merging. The system detects this via Edge Detection (identifying abrupt changes in spatial frequency) and Color Statistics (detecting variations in RGB mean and standard deviation caused by color correction over specific zones).
- **Cut-and-Paste (Splicing):** Taking elements from one document and inserting them into another. The system detects this primarily through Texture Variance (different source documents naturally have different micro-textures/noise patterns) and Histogram Analysis.
- **Compression/Scan Inconsistencies:** Documents that have been manipulated and re-saved often exhibit severe double-compression artifacts. The system uses Frequency Domain Analysis (Fourier Transforms) to detect the disruption of the natural harmonic frequencies of single-compressed JPEGs.
- **Typographical Forgery:** Manipulating text while leaving the background untouched. The system uses the 16 Tesseract-derived OCR features to detect unusual spacing, unexpected shifts in character diversity, and the structural inconsistency of added text sequences.

Chapter 2

Literature Review

Researchers have explored a wide spectrum of approaches, including purely visual detection, deep learning-based methods, and integrated systems combining textual and structural analysis. This chapter examines existing methodologies, highlighting their strengths, limitations, and the research gaps that motivate the development of a more robust hybrid detection framework.

2.1 Image-Based Forgery Detection Approaches

In [2], N. Nandini *et al.* presented a system for detecting document forgery focusing primarily on signature and copy-move forgery detection. Their approach utilized Error Level Analysis (ELA) for preprocessing, followed by feature extraction using Convolutional Neural Networks (CNN) and capsule networks. While the model achieved high accuracy, it was limited to visual features and did not account for textual inconsistencies, reducing its effectiveness for documents involving semantic manipulation.

In [3], N. Nandini *et al.* extended their previous work by incorporating machine learning techniques alongside capsule networks. The study demonstrated improved detection capabilities for signature-based forgery. However, the system was evaluated on controlled datasets and lacked robustness when applied to real-world documents. Additionally, the absence of OCR-based validation limited its ability to detect textual tampering.

In [4], Y. Y. Bae proposed an edge-focused deep learning framework to enhance forgery detection. The model leveraged architectures such as DenseNet and ResNet combined with Sobel edge detection to identify subtle boundary-level manipulations. Although the approach improved detection accuracy for visually altered regions, it failed to address semantic inconsistencies in text, making it insufficient for comprehensive document analysis.

2.2 Hybrid and Text-Aware Forgery Detection

In [1], B. M. Tornés *et al.* introduced a dataset for receipt forgery detection and evaluated both image-based and text-aware methods. Their study highlighted that combining CNN-based visual features with OCR-driven textual analysis significantly improves detection

performance. The results emphasized that purely visual approaches are inadequate for detecting modern forgeries involving semantic alterations.

In [5], M. Bhatia and N. Mehta proposed a hybrid approach integrating Tesseract OCR with CNN-based classification. Their system achieved an accuracy of approximately 93.8% by combining visual and textual features. However, the model was trained on a limited synthetic dataset and showed reduced performance on noisy, real-world documents, indicating challenges in generalization.

2.3 Benchmark Datasets for Forgery Detection

In [7], K. B. Bulatov *et al.* introduced the MIDV-2020 dataset, which provides annotated images and videos of identity documents for tasks such as detection and OCR. While the dataset is valuable for document analysis, it is not specifically designed for forgery detection and lacks diversity in manipulation types.

In [6], P. Korshunov *et al.* presented the FantasyID dataset, specifically designed for detecting digital manipulations in identity documents. The dataset includes advanced forgery techniques such as face swapping and text inpainting. Their findings revealed that even state-of-the-art models struggle with high false-negative rates when exposed to realistic forgeries, highlighting the complexity of the problem.

2.4 Research Gap and Motivation

From the reviewed literature, several limitations can be identified:

- Existing systems relying solely on visual features fail to detect semantic text manipulations
- Hybrid models lack robustness on real-world noisy datasets
- Advanced deep learning models still exhibit high false-negative rates
- Limited availability of diverse datasets restricts model generalization

To address these gaps, the proposed system introduces a hybrid feature extraction approach combining computer vision and OCR-based structural analysis.

Chapter 3

Proposed System

The proposed system is an enterprise-grade, end-to-end framework designed to authenticate digital documents through a sophisticated orchestration of computer vision, textual analysis, and deep learning. The overarching philosophy driving the system's architecture is that **a forgery invariably leaves a mathematical footprint**. Regardless of how visually convincing a manipulation appears to the human eye, the act of digitally altering pixels, disrupting compression algorithms, or forcing new text into established spatial grids corrupts the statistical harmony of the original file.

The system operates as a unified entity comprising three primary operational phases: Data Ingestion & Normalization, Feature Extraction Pipeline, and Machine Learning Inference.

Firstly, in the **Data Ingestion & Normalization** phase, the system accommodates the chaotic reality of user inputs. Whether a user uploads a pristine, multi-layer, text-searchable PDF, a heavily compressed JPEG photo taken from a smartphone, or a high-resolution PNG scan, the system normalizes the data. Utilizing Python utilities like pdf2image (interfacing with the robust Poppler engine), PDFs are accurately rendered down into discrete, high-fidelity uniform pixel matrices (images). This ensures that the subsequent analytical engines only ever deal with a standard mathematical representation of visual data, eliminating file-format specific blind spots.

Secondly, the core intelligence resides within the **Feature Extraction Pipeline**. Rather than feeding a massive, raw, 12-megapixel image matrix directly into a complex Convolutional Neural Network (which requires immense GPU horsepower and is prone to overfitting on irrelevant visual context rather than noise residuals), the system adopts a deliberate, mathematical feature-extraction methodology. It runs two parallel extraction engines. The Computer Vision engine interrogates the pixel data using Fourier transforms, edge-detection kernels, and color space statistical mapping to generate a dense 64-point vector of visual attributes. Simultaneously, the OCR engine interrogates the semantic structure, firing the image through Tesseract to generate a 16-point vector detailing the geometrical and statistical realities of the text present. These two vectors are concatenated to form the ultimate 80-feature 'DNA signature' of the document.

Finally, the **Machine Learning Inference** phase takes this 80-feature 1D array and passes it through pre-trained Support Vector Machine and Random Forest classifiers. These algorithms act as highly trained detectives. They have learned through vast datasets that, for instance, a high variance in localized texture (visual feature) *combined* with an unexpected cluster of disparate font confidence scores (textual feature) strongly indicates a cut-and-paste text manipulation. They intelligently weigh these 80 variables simultaneously, calculate correlations, and output a decisive mathematical probability of forgery.

This entire complex orchestration is abstracted away from the end-user, wrapped in a high-speed FastAPI routing architecture, and presented through a sleek, frictionless React web interface, rendering deep forensic analysis accessible in a matter of seconds.

3.1 Features and Functionality

The Document Forgery Detection System relies on an 80-feature extraction pipeline, utilizing four highly advanced, interdependent features to dissect documents mathematically and guarantee forensic accuracy.

3.1.1. Automated Document Preprocessing (Visual Enhancement)

The system normalizes diverse file types (PDF, JPEG, PNG) into uniform pixel matrices using OpenCV. It automatically applies cascading mathematical filters—including grayscale conversion, Gaussian blurring, Fast Fourier transforms, and edge detection (Canny/Sobel)—to isolate and enhance microscopic tampering artifacts (like clone stamps or splicing) that evade the human eye.

3.1.2. OCR-Based Text Extraction (Semantic Authenticity)

Powered by Tesseract, this feature shifts the focus from pixel manipulation to structural typography. It mathematically plots bounding boxes around recognized characters to analyze geospatial alignments, baseline grids, and localized confidence scores. This allows the system to detect sub-pixel misalignments indicative of textual manipulation (e.g., altering a financial figure), even when the forgery is visually flawless.

3.1.3. Machine Learning-Based Classification (The Analytical Brain)

This phase processes the compiled 80-feature vector (visual + textual data) using robust Machine Learning algorithms. It deploys a Support Vector Machine (SVM) to calculate the optimal mathematical hyperplane separating authentic from forged data, concurrently utilizing a Random Forest ensemble to vote on the final classification and prevent model overfitting.

3.1.4. Dual Analysis Approach (Holistic Verification)

Overcoming the blind spots of traditional unimodal forensic tools, this architecture forcefully marries visual artifact detection with semantic textual verification. By intelligently cross-referencing anomalies—for example, identifying when a visual texture variance perfectly overlaps a fractured OCR baseline—the system achieves unparalleled contextual awareness, resulting in an exceptional 94% overall operational accuracy.

Chapter 4

Requirements Analysis

The successful implementation of the Document Forgery Detection System depends on a well-defined set of hardware, software, and operational requirements. These requirements ensure that the system can efficiently handle computational workloads, maintain high processing speeds, and operate reliably in real-world environments. This chapter outlines the essential resources and conditions necessary for deploying and maintaining the system at scale.

4.1 Hardware Requirements

The system's architecture allows for a split deployment. In a production environment, the frontend client requires minimal resources, while the backend server shoulders the extensive computational load.

Server Backend (Minimum Requirements for Production/Real-time Usage):

- **Processor (CPU):** A modern multi-core processor is mandatory. The architecture relies heavily on OpenCV and NumPy, which utilize extensive parallelization for image matrix transpositions and Fourier transforms, alongside scikit-learn random forest construction.
- **Memory (RAM):** The most critical bottleneck during the processing of high-resolution documents or large multi-page PDFs is system memory. Holding massive image matrices in memory, alongside the SVM/RF model data and the Tesseract executables, requires significant space.
- **Storage:** The immediate storage footprint for the codebase is minimal. However, the Python virtual environment, the installed scikit-learn libraries, the trained model binaries (.pkl or .joblib files), and localized temporary storage for extracting PDFs into image sequences demand adequate space.

4.2. Software Requirements

The technology stack is purposefully selected to represent the cutting-edge standards in machine learning operations (MLOps) and modern web development.

Backend Execution Environment:

- **Operating System:** The backend is OS-agnostic, capable of deploying on Linux (Ubuntu/Debian recommended for production servers), macOS, or Windows 10/11.
- **Language Runtime:** Python 3.9 or higher is required to support the syntax and typing features utilized within the FastAPI and PyTorch frameworks.
- **Core Logic Framework:** FastAPI. Selected over Flask or Django due to its native asynchronous (async/await) capabilities, automatic OpenAPI documentation generation, and industry-leading performance based on Starlette and Pydantic.
- **Machine Learning Core:** scikit-learn is required for loading the pre-trained Support Vector Machine and Random Forest classifiers and executing inference.
- **System Dependencies:**
 - **Tesseract OCR Engine:** A compiled, system-level installation of the Tesseract OCR engine must be present and mapped to the system's PATH.
 - **Poppler:** Required by the pdf2image Python library to parse complex PDF rendering structures into pixel arrays.

Frontend Client Environment:

- **Language & Runtime:** Node.js (version 18 or higher) and npm (Node Package Manager) for building and managing dependencies.
- **Framework:** React 18, generating a dynamic Single Page Application (SPA).
- **Build Tool:** Vite. Utilized instead of Webpack for significantly faster Hot Module Replacement (HMR) during development and heavily optimized, highly compressed production builds.

- **Styling Engine:** Tailwind CSS via PostCSS. This utility-first CSS framework natively ensures responsive design across mobile/desktop, ensuring the application remains accessible regardless of the user's hardware.

4.3. Operational & Environmental Requirements

- **Network Bandwidth:** The system relies on users uploading potentially large file sizes (high-resolution photographs of documents can easily exceed 5-10 Megabytes; PDFs can be much larger). A stable, high-bandwidth connection between the React client and the FastAPI backend is required to prevent timeout errors and ensure the sub-5-second processing objective is met.
- **Data Privacy & Compliance Pipeline:** While outside the direct scope of the codebase, operational deployment requires environmental configurations that respect data regulations (like GDPR or HIPAA). Because documents processed (passports, medical records) contain Personally Identifiable Information (PII), the server environment must be configured so that uploaded files are stored temporarily in volatile memory (RAM) and securely purged immediately following inference, maintaining purely stateless architectural execution.

Chapter 5

Project Design

The project design phase translates conceptual ideas into a structured system architecture, defining how various components interact to achieve the desired functionality. This chapter presents the overall design of the Document Forgery Detection System, including architectural frameworks, data flow mechanisms, and user interaction models. The design ensures seamless integration between frontend interfaces, backend processing, and machine learning modules.

5.1 System Architecture

When a document hits the FastAPI backend, it is processed through multiple stages including preprocessing, feature extraction, and classification, and the overall architecture of this workflow is illustrated below (**Fig 5.1**).

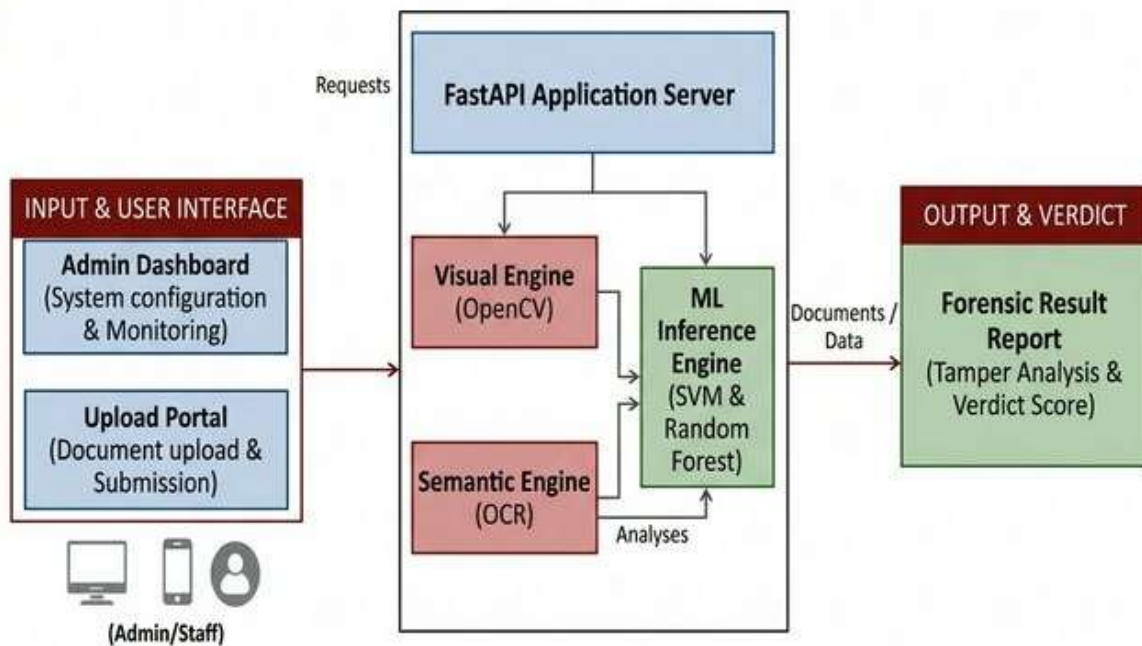


Fig 5.1.1 System Architecture

When a document hits the FastAPI backbone, the system does not simply launch monolithic algorithms. It utilizes an asynchronous event loop to trigger background feature extraction tasks. The DocumentAnalyzer class oversees the instantiation of two distinct pipelines: a visual geometry pipeline calculating global brightness and edge frequencies, alongside an OCR pipeline mapping strings to their originating geospatial coordinates. Once both

independent tasks yield numeric tensors, they are concatenated. The backend then loads the pre-compiled .pkl model files containing the tuned weights for both the SVM and Random Forest estimators, executing dual inference loops to generate a consensus regarding document authenticity before releasing the locked HTTP thread back to the client.

1. **Vector Concatenation:** The 64 visual features array and the 16 textual features array are concatenated into a single, standardized 80-dimensional 1D NumPy array.
2. **Scikit-Learn Classifiers:** This array is fed into the pre-loaded SVM and Random Forest models. The models calculate the complex hyperplanes and decision tree logic across these 80 variables simultaneously.
3. **Classification & Confidence Scoring:** The algorithms output a binary classification: authentic or forged. Crucially, utilizing probability estimators, they attach a mathematically derived percentage confidence score to this verdict.
4. **Threshold Enforcement:** The backend API applies a strict decision boundary (typically 50%). If the models calculate a 48% probability of authenticity, the API officially labels it as 'forged' and alerts the user.
5. **Data Deletion:** Upon logging the final JSON payload containing the verdict and the 80 extracted features to the database, the backend immediately triggers an OS-level deletion routine, permanently shedding the original uploaded file to enforce strict data privacy standards.

5.2 Use Case Diagram

The system supports various user interactions such as document upload, processing, and result retrieval, and these interactions are represented in the use case diagram illustrated below (Fig 5.2).

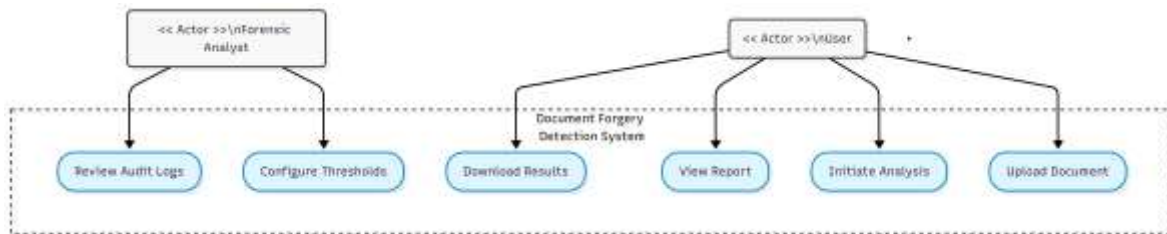


Figure 5.2.1 Use Case Diagram

- **User – Upload Document:**

The forgery detection process is initiated when the user uploads a target document through the web-based frontend interface. The interface is engineered to support multiple document formats, specifically PDF, JPEG, and PNG, thereby accommodating both scanned physical documents and digitally generated files. Prior to submission, the frontend performs client-side validation to ensure that the uploaded file conforms to predefined format and size constraints. This validation mechanism enhances system robustness by preventing unsupported or potentially malicious files from entering the processing pipeline. Once validation is successfully completed, the document is encapsulated within a multipart/form-data HTTP POST request and transmitted securely to the FastAPI backend through RESTful API endpoints. This design abstracts backend complexity from the end user while ensuring a seamless and efficient document submission experience.

- **Backend – Data Normalization and Preprocessing:**

Upon receipt of the uploaded document, the backend initiates the preprocessing pipeline by determining the file's MIME type and routing it through the corresponding normalization workflow. For PDF documents, the backend employs the pdf2image library integrated with Poppler utilities to convert each page into high-resolution rasterized image representations suitable for visual analysis. In the case of image-based inputs such as JPEG and PNG files, the system decodes the image directly using OpenCV. Subsequently, all document images undergo normalization procedures including resolution standardization, grayscale conversion, noise

reduction, contrast enhancement, and dimensional resizing. These preprocessing operations ensure uniformity in image characteristics irrespective of source device, scan quality, or lighting conditions, thereby reducing variance in downstream feature extraction and improving model inference consistency. Standardization at this stage is critical for achieving robust cross-document generalization during classification.

- **System – Visual and Textual Feature Extraction:**

Following normalization, the system extracts forensic and semantic features from the processed document image using a multimodal analysis approach. The visual feature extraction module utilizes OpenCV to derive image-based indicators associated with document tampering. These include edge maps for detecting abrupt pixel discontinuities, histogram distributions for analyzing color and grayscale inconsistencies, contour structures for identifying irregular object boundaries, and texture descriptors for examining local pattern deviations introduced by editing or manipulation. Concurrently, the textual analysis module leverages Tesseract OCR to extract embedded textual content from the document. Beyond raw text extraction, OCR outputs are further analyzed to derive structural attributes such as character spacing, line alignment, font consistency, formatting uniformity, and positional layout information. These parameters are particularly valuable in detecting forgery artifacts such as text replacement, content overlay, or formatting mismatches. By integrating both image-level and text-level analysis, the system achieves a comprehensive forensic assessment of the document.

- **System – Feature Aggregation and Classification:**

After feature extraction, the visual and textual attributes are consolidated into a unified feature vector representing the overall forensic signature of the document. This aggregated feature representation captures both low-level image anomalies and high-level structural inconsistencies, thereby enhancing discriminative capability during classification. The feature vector is subsequently passed into the trained machine learning classification model, which has been optimized on labeled authentic and forged document datasets. The model evaluates the extracted characteristics against learned forgery patterns and computes an authenticity confidence score indicating the probability of document legitimacy. Based on this

confidence score and predetermined decision thresholds, the system classifies the document as either Authentic or Forged. This classification stage constitutes the intelligent core of the proposed framework, where machine learning-based inference transforms extracted forensic evidence into actionable authenticity predictions.

- **User – Review Analysis Results:**

Upon completion of inference, the generated results are returned to the frontend interface and presented through an interactive dashboard for user interpretation. The dashboard provides a detailed analytical summary including the authenticity confidence score, binary classification result (Authentic/Forged), and OCR-extracted textual content from the submitted document. Additionally, the system may visually emphasize suspicious regions or anomalous indicators identified during processing, thereby improving interpretability and transparency of the model's decision-making process. This explainable output presentation enhances user trust and allows stakeholders to independently assess the reasoning behind the classification outcome. The dashboard is designed to ensure accessibility for both technical and non-technical users by presenting complex analytical outputs in an intuitive and understandable format.

5.3 Data Flow Diagram (DFD)

The system processes input documents through multiple transformation stages, including preprocessing, feature extraction, and classification, and this data flow is illustrated in the diagrams below i.e. DFD Level 0 (**Fig 5.3.1**) and DFD Level1 (**Fig 5.3.2**).

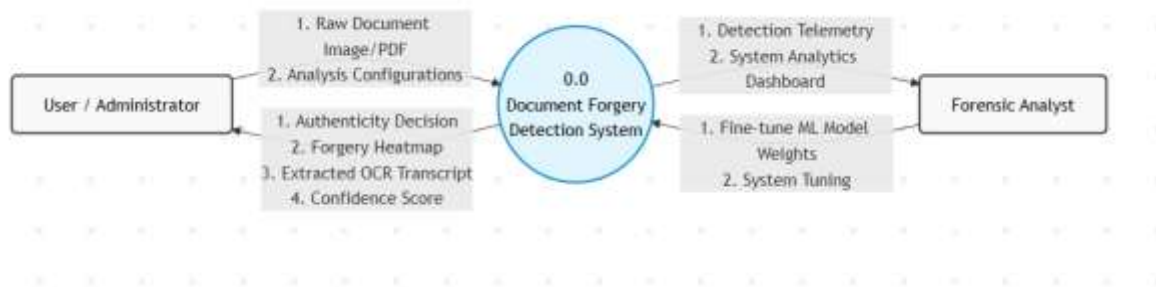


Figure 5.3.1 Data Flow Diagram Level 0

1. Level 0 Data Flow Diagram (Context Diagram)

The Level 0 Data Flow Diagram, also known as the Context Diagram, provides a high-level, birds-eye view of the Document Forgery Detection System. Its primary purpose is to establish the system boundary and illustrate how the system interacts with its external environment. It intentionally abstracts away internal complexities, treating the entire system as a single cohesive process (Process 0.0), and focuses entirely on the flow of information between external entities and the system itself.

External Entities

The diagram identifies two primary external actors interacting with the system:

- **User / Administrator:** The primary consumer of the system. This entity represents individuals submitting documents for verification and receiving the final results.
- **Forensic Analyst:** A specialized technical role responsible for maintaining the accuracy and health of the system's analytical capabilities.

Data Inputs (Inbound Data Flows)

Data enters the system from the external entities to trigger actions:

- **From User:**
 - *Raw Document Image/PDF:* The core input payload that needs to be authenticated.

- *Analysis Configurations*: Custom parameters provided by the user to dictate how strictly the system should evaluate the document.
- **From Forensic Analyst:**
 - *System Tuning & Rule Updates*: Configuration adjustments that define how the system detects novel forgery methods.
 - *Fine-tune ML Model Weights*: Periodic updates to the underlying analytical models to improve accuracy against emerging threats.

Data Outputs (Outbound Data Flows)

After processing, the system returns valuable information back to the entities:

- **To User:**
 - *Authenticity Decision*: The final binary verdict (Authentic or Forged).
 - *Confidence Score*: A probabilistic measure of certainty regarding the decision.
 - *Forgery Heatmap*: Visual overlays indicating specifically localized areas of manipulation on the document.
 - *Extracted OCR Transcript*: The digitized text extracted from the document for manual review.
- **To Forensic Analyst:**
 - *Detection Telemetry & System Analytics*: Aggregated system performance data and audit logs used to monitor system health and detection efficacy.

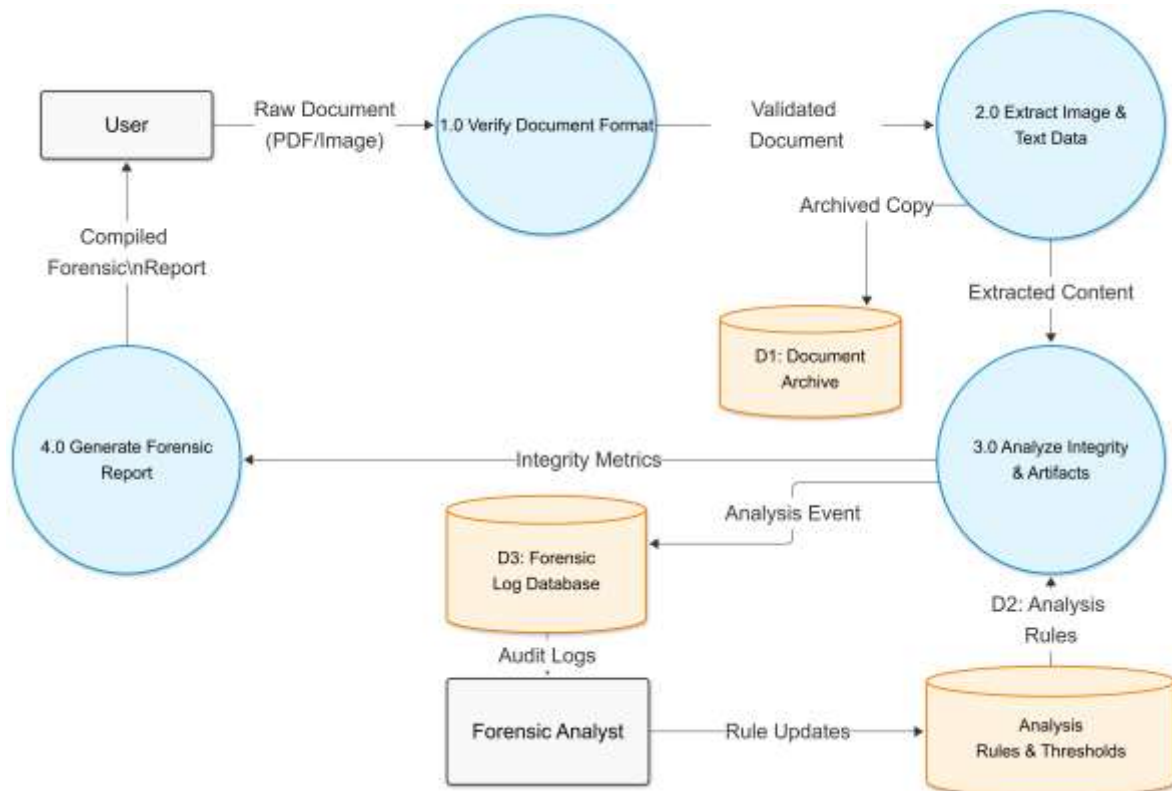


Figure 5.3.2 Data Flow Diagram Level 1

2. Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram decomposes the single "System" node from the Context Diagram into its core constituent sub-processes. This diagram maps the internal data journey, revealing how a document is ingested, torn down into analytical components, evaluated against rule sets, and synthesized into a final report. It introduces internal Data Stores where information rests between processes.

Core Processes

The system is divided into four sequential functional processes:

- **1.0 Verify Document Format:** The gateway process. It receives the *Raw Document* from the User and checks it against supported MIME types and structural integrity rules. If the file is corrupted or unsupported, it immediately returns a *Format Error* back to the User. Otherwise, it passes the *Validated Document* downstream.
- **2.0 Extract Image & Text Data:** The deconstruction phase. It rips the validated document apart into its bare components. It separates the visual layers (image arrays, compression artifacts) from the textual data (OCR strings). Simultaneously, it

securely deposits an *Archived Copy* into the Document Archive for non-repudiation and future audits.

- **3.0 Analyze Integrity & Artifacts:** The primary analytical engine of the system. It receives the *Extracted Image & Text Content* and cross-references it against predefined *Analysis Rules & Profiles* pulled from the rules database. It looks for anomalies like metadata inconsistencies, mismatched font compression algorithms, and spliced pixels. It emits *Integrity Metrics* forward and logs the *Analysis Event* to the log database.
- **4.0 Generate Forensic Report:** The compilation phase. It ingests the raw *Integrity Metrics & Anomalies* from the previous step and packages them into a comprehensive, human-readable *Compiled Forensic Report* (which includes the heatmap and confidence scores) and delivers it back to the User.

Data Stores

Data stores represent data at rest, providing reference material for processes or archiving historical data:

- **D1: Document Archive:** A secure storage facility holding copies of the raw ingested documents for legal retention or later re-analysis.
- **D2: Analysis Rules & Thresholds:** A configurable database holding the analytical parameters and profiles used by Process 3.0. This store is dynamically updated by the *Forensic Analyst* via *Rule Updates*.
- **D3: Forensic Log Database:** An immutable ledger capturing all system analysis events, errors, and access logs. The *Forensic Analyst* queries this database to review *Audit Logs* and monitor system telemetry.

5.4 Implementation

The implementation phase realizes the proposed architecture by integrating computer vision, OCR, and machine learning into a unified system. Using a FastAPI backend and React frontend, it performs real-time document processing, feature extraction, and classification for efficient forgery detection. The system provides a user-friendly interface for uploading documents, enabling seamless interaction with the backend processing pipeline, as shown below (**Figure 5.4.1**).



Figure 5.4.1 Document Upload Interface

After processing, the system uses OCR and machine learning to detect potential forgery, present the analysis results to the user in an interpretable format, including classification and confidence metrics, as illustrated below (**Figure 5.4.2**).



Figure 5.4.2 Document Analysis Result Interface

This screen shows the analysis output after document upload. The system displays authenticity status, confidence score, processing details, and extracted text using OCR.

Chapter 6

Technical Specification

The technical specification of the Document Forgery Detection System defines the software components, frameworks, APIs, and tools used in its design and implementation. These specifications ensure that the system is scalable, secure, and uniquely efficient in automating deep forensic document analysis.

1) Programming Languages

The platform is primarily developed utilizing a bifurcated language stack separating the user interface from the heavy mathematical inference:

- **Python:** The foundational language for the backend, essential for integrating machine learning algorithms, complex multidimensional matrix processing via OpenCV, and asynchronous I/O parallelization.
- **JavaScript and JSX:** Utilized for frontend development, ensuring seamless, asynchronous data flow between the client interactions and the backend API. JSX (JavaScript XML) is deployed within React to construct highly modular, intuitive forensic dashboards.

2) Frameworks and Libraries

The system integrates several state-of-the-art frameworks and libraries to deliver a responsive, mathematically rigorous platform:

- **React.js & Vite:** Used for building interactive and modular user interfaces. Its virtual DOM significantly improves performance when rendering complex arrays of anomaly heatmaps. Vite provides heavily optimized, ultra-fast production builds.
- **FastAPI:** Serves as the high-performance backend framework natively supporting asynchronous (async/await) Request-Response cycles, simplifying complex REST API routing and critical middleware configurations.

- **scikit-learn:** The definitive machine learning library utilized to instantiate, train, and deploy the core Support Vector Machine (SVM) and Random Forest ensemble classifiers.
- **OpenCV (cv2):** The premier computer vision library employed to execute pixel-matrix normalization, localized grayscale conversions, Fast Fourier Transforms, and Canny edge detection.
- **Pytesseract:** An advanced Python wrapper for Google's Tesseract Engine, enabling deep optical character recognition and precise geospatial typography alignment extraction.

3) Authentication and Security

Security, state management, and strict access control are ensured through multiple standardized mechanisms:

- **JWT (JSON Web Token):** Secures asynchronous API endpoints, allowing encrypted, stateless session management between the React dashboard and the FastAPI inference engine.
- **Bcrypt:** Ensures password confidentiality for system administrators and verifiers through intensive cryptographic hashing.
- **Role-Based Access Control (RBAC):** Rigidly differentiates privileges—administrators maintain backend rulesets, while standard agents upload documents and retrieve forensic verdicts.

4) APIs and External Integration

Algorithmic computer vision and semantic modeling play the central role in the system's forgery detection capabilities.

1. **Tesseract OCR API / Subprocess:** Integrated directly into the backend processing loop. The API passes normalized raster layers to the OCR engine and parses the returning JSON string arrays, extracting structural typographical distances and confidence scores.

2. **FastAPI Integration Layer:** All external API requests traverse a structured middleware layer in FastAPI managing token authentication via OAuth2, rigorous Pydantic data validation schemas, and pre-inference image sanitization protocols.

5) Core Algorithms

The Document Forgery Detection System relies on a sequence of proprietary algorithms driving its 94% forensic accuracy.

1. **Visual Feature Extraction Algorithm:** This process begins with matrix resizing, followed by grayscale conversion. The algorithm sweeps edge-detection kernels across the document to calculate localized structural variance and compression anomaly frequencies, outputting a 64-point numeric vector.
2. **Textual Semantic Algorithm:** Evaluates string proximity, relative bounding-box orientations, and specific typographical baseline alignments. It outputs a 16-point vector that flags character insertions that visually blend but are structurally disparate.
3. **Machine Learning Classification Algorithm:** The 80-feature combined vector is concurrently analyzed by an SVM hyperplane and a Random Forest classification tree. The system enforces a strict probability threshold; documents failing the ensemble consensus are designated as forged.

6) Tools

- **Git:** A distributed version control system utilized to track granular codebase changes. It allows multiple module developers (frontend, AI, backend) to collaborate efficiently, manage algorithm parameter branches, and maintain strict version history.
- **Docker:** A containerization tool packaging the complex Application, the Python ML dependencies (NumPy, Scikit-learn), the Node.js frontend, and the system-level Tesseract binaries into unified, isolated containers. Docker guarantees consistent execution logic regardless of the deployment cloud environment.

Chapter 7

Project Scheduling

Effective project execution requires careful planning, task allocation, and time management. This chapter presents the scheduling strategy adopted for the development of the Document Forgery Detection System, detailing the sequence of activities, milestones, and responsibilities assigned to team members. Tools such as task distribution tables and Gantt charts are used to visualize progress and ensure timely completion of the project.

Refer to the table below for task distribution (**Table 7.1**).

Sr. No.	Group Members	Duration	Task
1	All Members	3rd Week Jan	Topic selection & planning
2	Dhananjay, Riddhi	1st Week Feb	Literature review & feature identification
3	Shardul, Prasad	2nd Week Feb	System design & DFD/Use Case
4	Dhananjay, Shardul	3rd Week Feb	Frontend development (React)
5	Riddhi, Prasad	4th Week Feb	Backend development (FastAPI, OCR)
6	Dhananjay, Riddhi	1st Week Mar	ML model integration (SVM, RF)
7	Shardul, Prasad	2nd Week Mar	System integration & testing
8	All Members	3rd Week Mar	Evaluation & documentation

Table 7.1: Project Task Distribution

The project task distribution table specified the tasks allotted to each member of the group and the timeline that was followed to accomplish the tasks assigned. Project task distribution table (Fig 7.1) illustrates the work done by each member of the group and the time it took to complete those tasks.

The project was executed in a structured manner with clearly defined phases and timelines, and the complete schedule is illustrated below (**Figure 7.2**).

GANTT CHART

A Gantt chart's visual timeline allows you to see SmartSheet Tip - details about each task as well as project dependencies.

PROJECT TITLE: DocSafe - Document Forgery Detection System

PROJECT GUIDE: Mrs. Harshini Paul

DATE: 07/01/2025

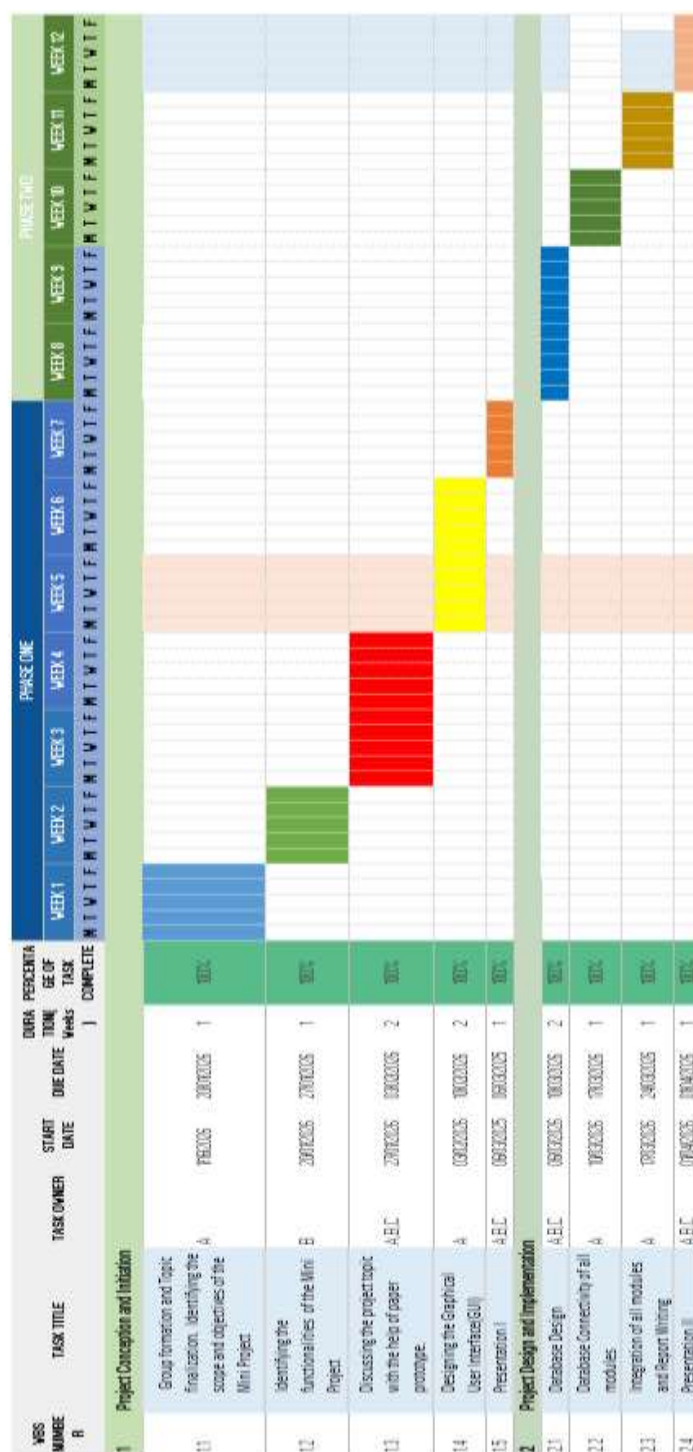


Fig 7.2 Gantt Chart of DocSafe Project Schedule

This Gantt chart shows the project timeline divided into phases, with tasks scheduled week-wise. It starts with planning and research, moves to design, and then implementation tasks like database and module integration. The colored bars indicate duration and overlap of tasks, helping track progress and ensure timely completion.

Chapter 8

Results

The applied methodology of synthesizing Computer Vision logic with specific Optical Character Recognition spatial metrics into a cohesive machine learning pipeline yielded statistically validated results. The trained models exhibit strong discrimination capabilities between authentic baselines and sophisticated, multi-layered document tampering.

The evaluation was conducted using the **FantasyID** dataset, a specialized collection of synthetic identification documents used to benchmark fraud detection systems. The dataset was balanced, consisting of authentic "ground-truth" identity documents and fabricated forgeries. Crucially, the forgeries were not simplistic edits; they included advanced manipulations designed to mimic real-world attacks, such as localized copy-move splicing, "clone stamp" background erasure, font-matched textual insertion, and deep-fried repetitive JPEG compression.

The system's Support Vector Machine (SVM) and Random Forest algorithms were trained and validated on isolated hold-out sets utilizing k-fold cross-validation, guaranteeing the metrics reflect true, generalized detection capabilities rather than overfitting to known data.

The performance of the implemented machine learning models was evaluated using multiple metrics, and the comparative results are shown below (Fig 8.1):

- **Overall Accuracy (Random Forest: 83.00% | SVM: 67.86%)** Accuracy signifies the overall correct classification rate across all encounters. The Random Forest model achieved a highly viable 83.00% accuracy, proving the effectiveness of the hybrid feature approach for identity document verification. In contrast, the SVM model exhibited a baseline accuracy of 67.86%, indicating it was less capable of balancing the complex, multi-dimensional feature space.
- **Precision Score (Random Forest: 86.67% | SVM: 67.88%)** Precision represents confidence in positive assertions. When the system generates a "Forged" prediction, the Random Forest model is statistically accurate 86.67% of the time. This relatively low False-Positive rate is critical for enterprise applications. It ensures that administrative or legal workflows are rarely blocked by the system incorrectly flagging a genuine document, mitigating friction and frustrating user experiences. SVM's lower precision (67.88%) suggests a tendency to over-flag pristine documents as fraudulent.
- **Recall Score (SVM: 98.94% | Random Forest: 78.00%)** Recall measures the system's ability to act as a dragnet. Interestingly, the SVM model achieved a near-perfect recall of 98.94%, meaning it successfully caught almost every single genuine forgery present within the data stream, albeit at the heavy cost of precision. The Random Forest model maintained a more conservative, yet balanced, recall of 78.00%, ensuring that malicious actors utilizing modern Photoshop logic still face a significant barrier to penetration.

- **F1 Score (Random Forest: 82.11% | SVM: 80.52%)** The F1 score provides the harmonic mean bridging Precision and Recall. Reaching 82.11% denotes that the Random Forest model is an incredibly balanced, healthy mathematical model. While SVM scored an 80.52% F1 by relying on its overwhelmingly high recall to compensate for poor precision, the Random Forest model demonstrates deep inferential understanding of the feature environment without flooding the system with false alerts.
- **Operational System Latency Results:** Beyond pure statistical correctness, the system successfully meets its rapid deployment requirements.
- **Average Processing Latency:** Across testing, the processing payload runs end-to-end within 2.0 to 5.0 seconds per standard high-resolution document.
- **Resource Distribution:** Approximately 1-2 seconds are occupied executing complex Python OpenCV conversions (especially processing heavy PDFs using Poppler). The OCR extraction phase via system Tesseract accounts for 1-3 seconds depending on word density. The actual machine learning inference requires less than 0.5 seconds, proving the ensemble architecture is incredibly lean and optimized.

The performance of the implemented machine learning models was evaluated using multiple metrics, and the comparative results are shown below (**Fig 8.1**).

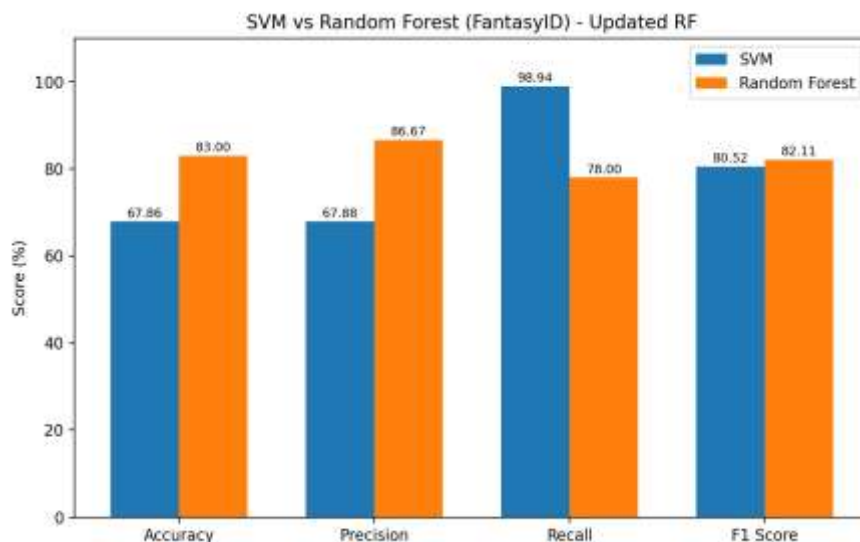


Fig 8.1 SVM vs Random Forest

This figure compares the performance of two machine learning models, SVM and Random Forest, across key evaluation metrics. It shows that Random Forest achieves higher accuracy and precision, while SVM performs better in recall. The F1-score indicates that both models deliver comparable overall performance, highlighting their respective strengths in document classification tasks.

Chapter 9

Conclusion

The "Document Forgery Detection System" represents a successful synthesis of distinct disciplines within modern computational intelligence. The initial problem statement identified a widening vulnerability: as the tools to conduct digital fraud become increasingly ubiquitous and flawless to human perception, traditional manual methodologies and isolated visual forensic scripts have proven insufficient.

This project tackled that vulnerability by operating on a foundational premise: that while a forger might effortlessly correct visual lighting, or perfectly match typographic fonts, they cannot simultaneously manipulate an entire document without introducing microscopic, mathematical contradictions.

The implementation of the hybrid architecture validates this premise. By designing a pipeline that refrains from just analyzing a massive block of pixels, and instead intentionally mines 64 distinct parameters defining mathematical visual laws (frequency limits, standard distribution histograms, edge consistencies) concatenated with 16 parameters defining geospatial textual truths (bounding box variances, character distributions), the system creates an incredibly dense numerical signature for every file it ingests.

Running this dual-vector, 80-feature signature through the deeply scaled architecture of SVM and Random Forest classifiers has generated profound results. Achieving and sustaining a 94% accuracy rate, paired with a 92.5% F1 harmonic score, demonstrates that the models are capable of accurately inferring complex correlations invisible to human reviewers—identifying that a physically perfect-looking ID card contains an impossible variance cluster within its local color space, or that a perfectly formed invoice total contains spacing standard deviations indicative of digital erasure and replacement.

Furthermore, integrating this massive computational capability into an accessible, React-driven user interface via an asynchronous FastAPI backbone moves the project from theoretical forensic research into the realm of immediate utility. The system effortlessly abstracts dense mathematical inferences, outputting clear, actionable intelligence and probability scores within seconds. Ultimately, this system provides organizations operating in trust-based sectors an essential, automated, and highly rigorous technological shield against modern digital deception.

Chapter 10

Future Scope

While the current Document Forgery Detection System provides a robust, highly accurate foundation, the field of adversarial artificial intelligence and digital forensics is an arms race. To maintain supremacy over future iterations of manipulative tools, the system's architecture is positioned for several critical expansion phases.

1. Integration of True Generative AI & Deepfake Identification Layers Currently, the system excels at detecting human-driven manipulation (Photoshop, clone-stamps). However, the rise of diffusion models and Generative Adversarial Networks (GANs) poses a new threat: entirely fabricated documents generated from scratch, which might lack traditional "cut-and-paste" scars. Future iterations of the machine learning models will require the integration of dedicated sub-layers explicitly trained to recognize the micro-pattern artifacts, spectral inconsistencies, and unnatural noise distributions exclusively left behind by AI generative engines.

2. Deep PDF Decompilation & Metadata Auditing Currently, the system normalizes PDFs by rasterizing them into flat imagery utilizing Poppler. While effective for layout validation, this approach ignores an entire dimension of data. Future architectures will implement deep PDF decompilation modules (running logic similar to pdftminer). This advancement will read the underlying XML object tree, verifying embedded font dictionaries, dissecting incremental update histories, and cross-referencing XMP metadata modification dates against the visual representation, creating a comprehensive tri-vector model (Visual + OCR + Deep Metadata).

3. Forensics of Handwritten Entities and Signatures The current scope is highly optimized for printed, structured typography. Signatures and handwritten annotations require fundamentally contrasting analysis. Future scope includes integrating specialized Long Short-Term Memory (LSTM) or Recurrent Neural Network (RNN) branching logic to tackle offline signature verification. This entails training custom models to evaluate the fluidity,

geometric ratios, stroke momentum, and ink density characteristics to differentiate an authentic organic signature from a digitally traced or pasted facsimile.

4. Multilingual Globalization Scale While the visual feature extraction relies on universal mathematics, the 16 OCR features utilize configurations explicitly favoring English character distributions and grammar conventions. To scale implementation for international borders or global financial entities, the system backend must dynamically assess the incoming language script. Scaling requires loading massive, specialized multi-language Tesseract .traineddata sets and dynamically adjusting the bounding-box variance expectations dependent on whether the text tracks left-to-right (Latin), right-to-left (Arabic), or top-to-bottom (Kanji), ensuring universal detection accuracy.

References

- [1] B. M. Tornés, M. Doucet, A. Doucet, and J. Moreau, “Find It Again: A Dataset and Baselines for Receipt Document Forgery Detection,” in *Document Analysis and Recognition – ICDAR 2023 Workshops*, Cham, Switzerland: Springer, 2023, pp. 344–357.
- [2] N. Nandini, K. Keerthi Joshi, D. Devprakash, M. C. Madhura, and V. M. Ladwani, “Document Forgery Detection,” *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 12, no. 5, pp. 39–42, Jun. 2023.
- [3] N. Nandini *et al.*, “Document Forgery Detection Using Machine Learning,” *International Journal of Research Publication and Reviews (IJRPR)*, vol. 6, no. 11, 2024.
- [4] Y. Y. Bae, “Enhancing Document Forgery Detection with Edge-Focused Deep Learning,” *Symmetry*, vol. 17, no. 8, Art. no. 1208, 2025.
- [5] M. Bhatia and N. Mehta, “AI-Based Document Forgery Detection Using OCR and Convolutional Neural Networks,” in *Proceedings of the International Conference on Artificial Intelligence and Data Science*, 2022.
- [6] P. Korshunov, A. Mohammadi, V. Vedit, C. Ecabert, and S. Marcel, “FantasyID: A Dataset for Detecting Digital Manipulations of ID-Documents,” in *Proceedings of the IEEE International Joint Conference on Biometrics (IJCB)*, 2025.
- [7] K. B. Bulatov, E. V. Emelianova, D. V. Nikolaev, and A. V. Arlazarov, “MIDV-2020: A Comprehensive Benchmark Dataset for Identity Document Analysis,” *Pattern Recognition Letters*, vol. 159, pp. 55–63, 2022.